# PRACTICAL - 12

## Implementation of Travelling Salesman Problem

**Aim of the Practical:**

To solve the Travelling Salesman Problem (TSP) using dynamic programming with bitmasking and analyze its exponential state space.

**Concept:**

The DP state dp[mask][u] stores the minimum cost to start at vertex 0, visit exactly the cities represented by mask, and finish at city u.

**Program:**

```
#include <iostream>
#include <vector>
#include <climits>
using namespace std;
int main() {
    vector<vector<int>> cost = {
        {0,10,15,20},
        {10,0,35,25},
        {15,35,0,30},
        {20,25,30,0}
    };
    int n = cost.size();
    int N = 1 << n;
    const int INF = 1000000000;
    vector<vector<int>> dp(N, vector<int>(n, INF));
    dp[1][0] = 0;
    for (int mask = 1; mask < N; mask++) {
        for (int u = 0; u < n; u++) {
            if (!(mask & (1 << u)) || dp[mask][u] == INF) continue;
            for (int v = 0; v < n; v++) {
                if (mask & (1 << v)) continue;
                int nextMask = mask | (1 << v);
                dp[nextMask][v] =
                    min(dp[nextMask][v], dp[mask][u] + cost[u][v]);
            }
        }
    }
    int ans = INF;
    int full = N - 1;
    for (int u = 1; u < n; u++)
        ans = min(ans, dp[full][u] + cost[u][0]);
    cout << "Minimum tour cost = " << ans;
}
```

**Output:**

```
Minimum tour cost = 80
```

**Time Analysis:**

**Step-by-Step Derivation:**

Let n be the number of cities.

A bitmask has 2^n possible subsets of cities.

For each subset mask, the DP stores a value for each possible current city u.

$$\text{Number of DP states} = n \cdot 2^n$$

For each state, the transition may try up to n unvisited next cities.

$$\text{Work per state} = O(n)$$

$$T(n) = O(n \cdot 2^n \cdot n)$$

$$T(n) = O(n^2 \cdot 2^n)$$



Name: KUMMARI SHIVA TEJA Enrollment No: 92460118182

The DP table contains n·2^n entries.

$$S(n) = O(n \cdot 2^n)$$

This is substantially better than checking all n! tours by brute force, but it is still exponential.

**Final Time Analysis:**

- Time Complexity: $O(n^2 \cdot 2^n)$.
- Space Complexity: $O(n \cdot 2^n)$.

**Result:**

The dynamic-programming TSP implementation found a minimum tour cost of 80 for the sample graph.